Yazılım Mühendisliği ve Yazılım Süreçleri

Kodlamadan önceki sorular.

İhtiyacı anla

Gereksinimi tanımla

Çözümü kontrol et

Önce doğru problemi anlayalım.

Hata vermeyen bir program, başarılı bir yazılım mıdır?

Çalışıyor mu?

Teknik davranış

İhtiyacı karşılıyor mu?

Kullanıcı amacı

Kullanılabiliyor mu?

Gerçek kullanım

Çalışması, doğru ihtiyacı karşıladığı anlamına gelmez.

Bugün üç adımda ilerleyeceğiz

01 Mühendislik ve süreç

02 Yaklaşımlar ve gereksinimler

03 Bir sistemi birlikte çözümleme

Neyi geliştireceğiz? Kararlarımızı nasıl kontrol edeceğiz?

Yazılım mühendisliği: üç anahtar

Yazılımı geliştirmek, işletmek ve sürdürmek için bir yaklaşım.

SİSTEMATİK

Düzen

İşleri tanımlı bir akışla yürütürüz.

DİSİPLİNLİ

Takip

Kararları ve değişiklikleri izleriz.

ÖLÇÜLEBİLİR

Kanıt

Sonucu ölçütlerle kontrol ederiz.

Sadece üretmek değil; gerekçelendirmek ve kontrol etmek.

Kodlama, bütünün bir parçasıdır

YAZILIM MÜHENDİSLİĞİ

İhtiyaç › Tasarım › **Kodlama** › Test › Bakım

Amaçlar, kararlar, kalite ve değişiklikler birlikte ele alınır.

Kodun doğru olması, yanlış seçilmiş problemi düzeltmez.

Üç açıdan düşünürüz

İNSAN	Kimin ihtiyacını karşılıyoruz?
ÜRÜN	Hangi davranış ve kalite bekleniyor?
SÜREÇ	Kim, neyi yapacak ve kontrol edecek?

İyi bir çözüm, yalnızca teknoloji seçimiyle başlamaz.

Kaynak kodu vermek yeterli mi?

Projeyi yarın başka bir ekip devralıyor.

Kaynak kod

Çözümün gerçekleştirilmiş hâli

+

Amaç ve kurallar
Kararlar ve testler
Kurulum ve kullanım

Yazılımın yanında, onu anlamayı sağlayan bilgi de gerekir.

Koddan önce ve sonra bir yaşam var

Fikirden kullanıma; bakımdan kullanım dışı bırakmaya.

01
Gereksinim
Ne yapılmalı?

02
Tasarım
Nasıl kurulmalı?

03
Geliştirme
Nasıl gerçekleştirilir?

04
Kontrol
Beklentiye uygun mu?

05
Kullanıma alma
Ortama hazır mı?

06
Bakım
Neler değişmeli?

Bu çalışmalar gerektiğinde tekrar edilir ve birbirini besler.

Bir süreç dört soruyu açıklaştırır

KİM?

Sorumluluk kimde?

NE YAPACAK?

Hangi iş yürütülecek?

NE ÜRETECEK?

Beklenen çıktı ne?

NASIL KONTROL?

Sonuç nasıl değerlendirilecek?

Süreç, belirli bir sonuca götüren ilişkili çalışmalardır.

Ortak bir dil: ISO/IEC/IEEE 12207

12207

Yaşam döngüsü süreçleri

1. Ortak bir süreç çerçevesi sunar.
2. Geliştirme, işletim ve bakımı kapsar.
3. **Tek bir geliştirme modeli dayatmaz.**

Standart, tek bir zorunlu proje tarifi değildir.

İlerlemek, geri dönmemek değildir



İnceleme ve test yalnızca sona bırakılmaz.

Yeni bilgi, önceki kararları değiştirebilir.

Bir değişiklik, birden fazla etki



Değişikliği değil, etkilerini de yönetiriz.

Aynı işler, farklı çalışma düzenleri

AŞAMA AĞIRLIKLI

Sırayla ilerleme

Aşama çıktıları üzerinden düzen

ARTIMLI / YİNELEMELİ

Büyütme ve iyileştirme

Yeni yetenekler ve geri dönüşler

ÇEVİK

Geri bildirim ve uyarlama

Kısa aralıklarla öğrenme

Bu yaklaşımlar, birbirini tamamen dışlayan kutular değildir.

Waterfall: aşama çıktılarıyla ilerlemek



Başlangıçtaki netlik önemlidir. Geç değişiklik: yeniden çalışma.

Bir aşamanın çıktısı, sonraki çalışmaya temel olur.

Incremental: ekle. Yineleme: iyileştir.

ARTIMLI

Yeni yetenek eklenir.



YİNELEMELİ

Mevcut çözüm iyileşir.

İlk çözüm › İyileşen çözüm

Geri bildirimle yeniden ele al.

Aynı projede hem genişletebilir hem iyileştirebiliriz.

Agile yaklaşım: geri bildirimle uyarlama

Küçük parça › Çalışan ürün › **Geri bildirim**

Çeviklik ≠ plansızlık

Çeviklik ≠ dokümansızlık

Plan yapılır; yeni bilgiyle güncellenir.

Üç soru: Neden? Ne? Nasıl?

NEDEN?

İhtiyaç

Çözülmesi gereken problem veya amaç

NE?

Gereksinim

Çözümün sağlaması gerekenler

NASIL?

Tasarım

Gereksinimi karşılayan çözüm yapısı

Ne yapılacağını netleştirmeden teknolojiye atlamayalım.

Gereksinim yalnızca özellik listesi değildir

İŞLEV	**Ne yapacak?**	Davranışlar ve hizmetler
KALİTE	**Ne kadar iyi?**	Performans, kullanılabilirlik
KISIT	**Hangi sınırlara uyacak?**	Zorunlu koşullar ve altyapı

Davranış + kalite beklentisi + zorunlu sınırlar.

Gereksinim mühendisliği ne yapar?

1. **DİNLE** — Paydaşları anla
2. **ANALİZ ET** — Eksik ve çelişki ara
3. **YAZ** — Açıkça belirt
4. **KONTROL** — İhtiyaçla karşılaştır
5. **YÖNET** — Değişimi izle

Söylenen isteği olduğu gibi kodlamakla yetinmeyiz.

İyi gereksinim için dört kontrol

AÇIK

Tek bir anlam taşıyor mu?

GEREKLİ

Bir ihtiyaca dayanıyor mu?

YAPILABİLİR

Kaynaklar ve koşullar yeterli mi?

DOĞRULANABİLİR

Sağlandığını gösterebilir miyiz?

“Hızlı” demek yetmez: hangi koşulda, ne kadar hızlı?

İki farklı kontrol sorusu

DOĞRULAMA

Gereksinime uygun mu?

GEÇERLEME

Gerçek ihtiyacı karşılıyor mu?

Doğru yapmak ile doğru şeyi yapmak aynı değildir.

Dönem boyunca hangi yolu izleyeceğiz?

Yazılım süreçleri ve gereksinim temelleri

Belirleme, kullanıcılar ve dokümantasyon

Ara sınav

Modelleme, kalite, prototip ve doğrulama

İhtiyaçtan dokümana, modele ve prototipe ilerleyeceğiz.

Dersin değerlendirme düzeni

YARIYIL İÇİ PUANI

%40	Ara sınav
%60	İki projenin toplam katkısı

DÖNEM BAŞARI PUANI

%50	Yarıyıl içi ortalaması
%50	Final

Yarıyıl içi ortalaması ve final, başarı puanına eşit katkı yapar.

Aklımızda üç cümle kalsın.

1. Kodlama, bütünün bir parçasıdır.
2. Gereksinim, çözümün ne sağlayacağını söyler.
3. Başarı, ihtiyacı karşılamakla ilgilidir.

Şimdi tek bir sistem üzerinde birleştirelim.

Kütüphane çalışma odası rezervasyonu

“Oda ayırabileceğimiz,
hızlı ve kolay bir uygulama istiyoruz.”

Kim?
Ne zaman?
Hangi kuralla?

Öğretim amaçlı senaryo; kurallar ve sayılar varsayımsaldır.

Kod yazmaya hazır mıyız?

Önce ihtiyacı ve kapsamı netleştirelim

İHTİYAÇ

Belirsizliği azalt

Uygun odayı önceden gör.
Kullanım karışıklığını azalt.

PAYDAŞLAR

Doğru kişileri dinle

Öğrenci
Kütüphane görevlisi
Bilgi işlem

İLK KAPSAM

Temel akışı kur

Oda görüntüle
Rezervasyon yap
İptal et

Bu sürümde yok: kitap ödünç alma ve ceza ödeme.

İstekleri kontrol edilebilir kurallara çevir

Varsayım: rezervasyonlar birer saatlik zaman dilimleridir.

G1	Sistem, seçilen zaman dilimindeki uygun odaları listelemelidir.
G2	**Aynı oda ve saat için en fazla bir rezervasyon aktif olmalıdır.**
G3	Öğrenci, kendi rezervasyonunu başlamadan iptal edebilmelidir.
G4	Bir öğrencinin günlük aktif rezervasyon sayısı ikiyi aşmamalıdır.

Her gereksinim için soralım: sağlandığını nasıl gösteririz?

“Hızlı olsun”u ölçülebilir hâle getirelim

Oda sorguları • tanımlanmış test ortamı ve veri büyüklüğü

100	%95	≤ 2 sn
eş zamanlı kullanıcı	sorguların en az bu oranı	yanıt süresi

Kısıt: Üniversitenin kimlik doğrulama hizmeti kullanılacak.

Test yükü ve eşikler, gerçek ihtiyaçla gerekçelendirilmelidir.

Prototip: koddan önce ekran akışı

Çalışma odası ayır

Yarın • 14.00–15.00

Oda A	**Uygun**
Oda B	Dolu

Oda A’yı seç

›

Rezervasyonu onayla

Oda A

Yarın, 14.00–15.00

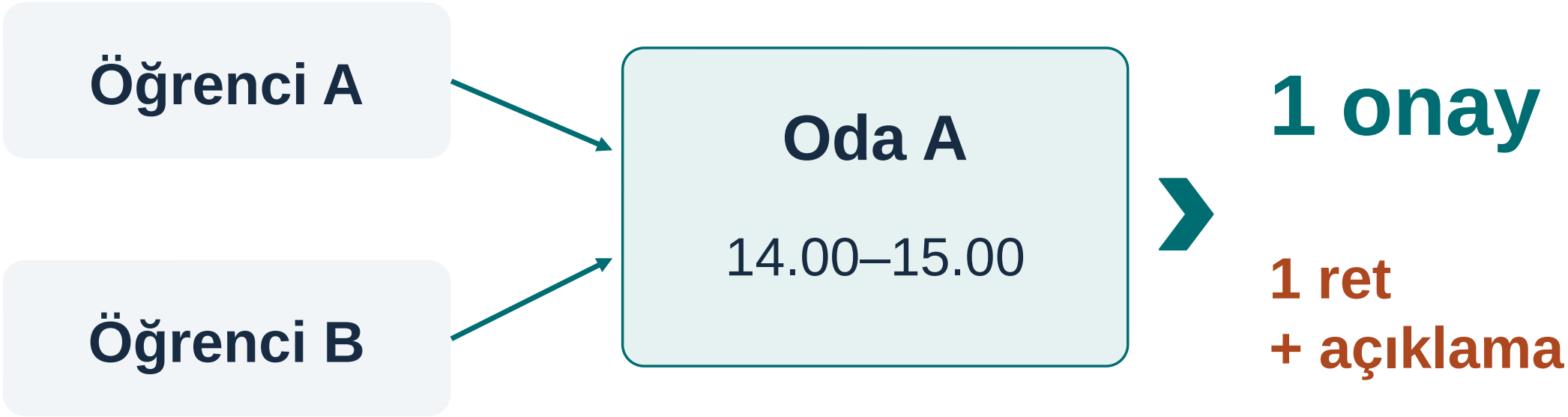
Geri dön

Onayla

Öğrenci, işlemi ek açıklama olmadan tamamlayabiliyor mu?

İki öğrenci aynı anda aynı odayı isterse?

Başlangıç durumu: oda bu zaman diliminde boş.



Kabul ölçütü: bu oda ve zaman diliminde tek aktif rezervasyon.

Kurallara uydu. Peki sorun çözüldü mü?

YENİ GÖZLEM

Rezervasyon var.
Öğrenci gelmiyor.
Oda boş kalıyor.

HENÜZ KARAR VERMEDİK

İptal süresi mi?
Giriş onayı mı?
Başka bir kural mı?

Kararı paydaşlarla netleştir; belge, kod ve testleri gözden geçir.

Gereksinime uygunluk, ihtiyaca uygunluğu garanti etmez.

Bir düğmenin arkasında bir sistem var.

İHTİYAÇ	KURAL	KANIT	GERİ BİLDİRİM
Oda karmaşasını azalt	Aynı odada çakışmayı önle	Eş zamanlı istekleri dene	Gerçek kullanımı izle

Önce ihtiyacı anla.
Sonra ne yapılacağını tanımla.